



6. Clase 6

Clase N° 6: Lenguajes de programación para Ciencia de Datos; fundamentos de Procesamiento de datos y Ciencias de Datos con R

Contenido:

?

En la clase de hoy trabajaremos los siguientes temas:

- Introducción al Lenguaje R
- Objetos y operaciones en R
- Listas, dataframes en R
- Lectura de archivos
- Data frames en R, filtrado y ordenamiento
- Análisis estadísticos descriptivos en R

1. Presentación

Hola y bienvenidos al curso de Procesamiento de datos.

En esta sexta semana de clases comenzaremos a estudiar el Lenguaje R, introduciendo sus características y su sintáxis básica. Aprenderemos a manipular objetos, definir estructuras de datos como listas y dataframes y leer archivos externos. Te invito a ver el siguiente video <https://www.youtube.com/watch?v=3hR2A2nCI4U> “¿Qué es R y para qué nos sirve?” para introducirte en el tema.

2. Desarrollo y actividades

¿Qué es el Lenguaje R?

R es un software matemático que fue desarrollado inicialmente por Robert Gentleman y Ross Ihaka del Departamento de Estadística de la Universidad de Auckland (Nueva Zelanda) en 1990. Sin embargo, el proyecto se inició en los Bell Laboratories de AT&T

en Nueva Jersey con el Lenguaje S en el año 1970.

Su desarrollo actual es responsabilidad del R Development Core Team (2019), cuyo Website se encuentra en: www.r-project.org

Principales características del lenguaje R

R es un lenguaje y entorno de programación para análisis estadístico y gráfico. Se trata de un proyecto de software libre, resultado de la implementación del Lenguaje S. Probablemente, R es el lenguaje más utilizado en investigación por la comunidad estadística. A esto contribuye la posibilidad de cargar diferentes Librerías o Paquetes con finalidades específicas de cálculo o gráfico.

R se distribuye bajo la licencia GNU GPL y está disponible para los sistemas operativos Windows, Macintosh, Unix y GNU/Linux.

R se trata de un lenguaje de programación que es muy potente y con una sintaxis muy sencilla e intuitiva de aprender. No se necesitan conocimientos de otros lenguajes de programación para poder usar R.

R proporciona un abanico de herramientas estadísticas (e.g., modelos lineales y no lineales, test estadísticos, algoritmos de clasificación y agrupamiento, etc.) y gráficas.

R puede integrarse con distintas bases de datos y existen librerías que facilitan su utilización

R permite generar gráficos con alta calidad

R también puede usarse como herramienta de cálculo numérico y puede ser tan eficaz como otras herramientas tales como MATLAB.

R forma parte de un proyecto colaborativo y abierto. Sus usuarios pueden publicar Librerías (también denominadas paquetes, del inglés package) que extienden su configuración básica. Las librerías están organizadas por temas.

En resumen, las ventajas del lenguaje R son las siguientes:

- R es un lenguaje de programación eficaz que permite hacer todo tipo de análisis estadísticos.
- Aunque en un principio R puede resultar poco amigable, ya que requiere redactar códigos, su sintaxis es relativamente fácil de aprender.
- R permite combinar fácilmente librerías de código R y de otros programas y ofrece gráficos de alta calidad.
- R es gratuito. Por tanto, se utiliza sin ningún coste. Esto hace que su alcance sea mucho mayor que el de cualquier otro programa comercial.
- R se está actualizando permanentemente, con las aportaciones de la comunidad científica de expertos en Estadística. Esto obliga al usuario a actualizar el programa de forma permanente también en su equipo.
- R tiene un Menú de Ayuda muy completo. Además, en la web pueden encontrarse múltiples tutoriales y videos donde se explica el manejo de muchas de las funcionalidades y librerías del programa.
- R no dispone de un soporte comercial pero se actualiza permanentemente por miles de usuarios en todo el mundo, lo que permite contactar online y consultar directamente con el autor o autora de la función o librería en cuestión, aquellas dudas que tengamos.

Crear objetos

Como ya se ha señalado, R es un lenguaje de programación y, como tal, se necesita definir todas las operaciones que queramos llevar a cabo. Para ello, necesitaremos manejar variables denominadas Objetos.

Los objetos pueden tener el contenido que queramos. Entre otros: Un dato

(por ejemplo, el número 2).

Múltiples datos (por ejemplo: 1, 2, 3, 4, etc.)

Nombres y cadenas (“Hola UAM”, como en el ejemplo de la Figura 1.12). Bases de datos (e.g., un fichero SPSS).

Conjuntos de bases de datos. etc.

Cada objeto recibe un nombre y es guardado en la memoria para ser reutilizado posteriormente. Por tanto, un paso importante a realizar implica “dar contenido al objeto”. Por ejemplo, si creamos el objeto x y queremos definir que $x = 10$, esto sería dar contenido a nuestro objeto (en este caso, un valor numérico). En R se utiliza una flecha `<-` o el signo `=` para crear objetos y darles contenido.

Veamos un nuevo ejemplo.

Crearemos los objetos a, b y suma:

```
a <- 1
```

```
b = 2
```

```
suma = a + b
```

a y b incluyen números. Para definir a, qué es el número 1, se ha utilizado la expresión `<-`; mientras que para definir b se ha utilizado el signo `=`. Ambos operadores son equivalentes. Teniendo definidos los objetos a y b, puede definirse el objeto suma que es la operación $a + b$. Si más adelante queremos utilizar el objeto suma, podremos llamarlo y que forme parte de otro objeto diferente. Podría, como en el ejemplo inferior referido al objeto c, ser el numerador de un cociente:

```
c <- suma / b
```

Debemos tener en cuenta que en R lo que no se guarde en un objeto, se pierde. Por tanto, es necesario mentalizarse de que el trabajo con R requiere estar permanentemente definiendo objetos.

También es importante tener en cuenta que R distingue entre letras mayúsculas y minúsculas. En el siguiente ejemplo, obsérvese que los objetos `d` y `D`, pese a tener la misma letra, son distintos:

```
d <- 8
```

```
D <- 9
```

Si usamos el mismo nombre para un objeto que ya existe, se sobrescribe. En el ejemplo, al ejecutar los siguientes comandos:

```
f <- 5
```

```
f <- 6
```

el objeto `f` termina tomando el valor 6

Las operaciones que pueden realizarse con objetos en R son muy diversas. Entre las que nos interesan se encuentran las siguientes:

Operación y Notación

Suma + Resta

- Producto *

División /

Exponente ^

Raíz cuadrada sqrt Producto

matrices %* %

A continuación, veremos ejemplos de sintaxis acerca de las operaciones de la tabla anterior y de sus resultados, que se muestran con el nombre del objeto seguido de la expresión `## [1]`, que es el resultado obtenido al ejecutar cada comando con Run:

```
a <- 1
```

```
b <- 3
```

```
r <- a + b
```

```
s <- b - a
```

```
p <- s*r
```

```
q <- sqrt(p) r
```

```
## [1] 4
```

```
s
```

```
## [1] 2
```

```
p
```

```
## [1] 8
```

```
Q
```

```
## [1] 2.828427
```

Vectores en R

Para crear un objeto o vector que contenga series de números, puede usarse el comando `c()`, que significa conjunto o concatenar. Los elementos incluidos en la serie numérica han de ir separados por comas.

En el ejemplo siguiente se crean dos tipos de series de objetos. Uno con nombres (denominado: nom) y otro con números (x):

```
nom <- c("Manuel","Eduardo")
```

```
x <- c(1,4,3,8)
```

También es posible crear una secuencia de números utilizando el comando 1:N. En el siguiente ejemplo, referido a num1 y num2, con cinco datos, podríamos definirlo de dos maneras que, como se observa, llevan al mismo resultado:

```
num1 <- c(1, 2, 3, 4, 5)
```

```
num2 <- 1:5 num1
```

```
## [1] 1 2 3 4 5
```

```
num2
```

```
## [1] 1 2 3 4 5
```

Los objetos también se pueden concatenar. Por ejemplo, podemos definir num3 a partir de num1 y num2 del siguiente modo:

```
num3 <- c(num1,num2) num3
```

```
## [1] 1 2 3 4 5 1 2 3 4 5
```

Para comprobar el tipo de datos de un vector puede usarse el comando str. Por ejemplo:

```
str (num1)
```

```
## num [1:5] 1 2 3 4 5
```

Listas

Una lista (list) es una colección ordenada de objetos conocidos como componentes. La lista se diferencia del vector en que los componentes pueden ser de diferentes tipos. Una lista puede estar compuesta por elementos de distintos tipos como valores numéricos, cadenas de caracteres, otra lista e incluso una matriz. Para acceder a los elementos de una lista se utilizan los corchetes [], y para acceder a los elementos de una lista que está dentro de otra lista se utiliza el doble corchete [[]].

En este ejemplo utilizamos la función list para crear una lista que contiene dos componentes, una cadena de texto y un vector numérico:

```
lista <- list("Valores de un dado", 1:6)
```

```
lista[1]
```

```
lista[2]
```

```
lista[[2]][3]
```

```
lista[[2]] > 3.5
```

Es posible unir varias listas en una mayor utilizando la función c(). Por ejemplo:

```
lista1 <- list("Valores de un dado", 1:6)
```

```
lista2 <- list("Valores de una moneda", c("cara", "cruz")) lista <-
```

```
c(lista1, lista2)
```

Dataframes en R

Los marcos de datos se crean con el comando data.frame(), en el que se indican el nombre y los valores de sus variables. Para acceder a las variables que forman parte de un marco de datos podemos utilizar tanto el símbolo \$ como el doble corchete [[]].

Además, podemos utilizar el comando `str()` para obtener información sobre la estructura de un marco de datos. Por ejemplo, el siguiente código crea un marco de datos con dos variables, una que contiene los nombres de los sujetos y otra su edad.

```
df <- data.frame(nombres <- c("Claudia", "Gador", "Alba"), edades  
<- c(8, 9, 12))  
str(df)  
df  
df$nombres  
df$edades df[[1]];  
df[[2]]
```

En ocasiones, la utilización del signo `$` puede ser engorrosa para referirse a las variables incluidas en un marco de datos. Para evitar utilizar `$` podemos llamar a la función `attach`, que hace que todas las variables incluidas en un marco de datos sean visibles.

Continuando con el ejemplo anterior:

```
df <- data.frame(nombres <- c("Claudia", "Gador", "Alba"), edades  
<- c(8, 9, 12))  
df attach(df)  
nombres  
edades
```

Leer datos de archivos externos

Los marcos de datos pueden leerse y guardarse en archivos de datos en formato texto, SPSS y Excel utilizando las funciones que veremos a continuación.

Datos de texto

Para leer datos en formato de texto se utiliza la función `read.table()`. Por ejemplo, podemos leer el archivo de datos `ejemplo.dat` en R utilizando los siguientes comandos:

```
ejemplo <- read.table("ejemplo.dat", header=TRUE) names(ejemplo)
```

Una matriz que hayamos creado en R podemos guardarla en un archivo de texto utilizando la función `write.table`. Por ejemplo, en las siguientes líneas de código definimos una matriz con tres filas y dos columnas y la guardamos en el archivo `matriz.dat`:

```
matriz <- matrix(c(1,2,3,4,5,6), nrow=3)
```

```
write.table(matriz, "matriz.dat", row.names=F, col.names=F) Datos CSV
```

El formato Comma Separated Values (CSV) es un tipo de archivo de texto en el que las variables están separadas por comas y los decimales por punto. En el formato CSV2 las variables se separan por punto y coma y los decimales por coma. Para grabar un marco de datos en estos formatos utilizaremos la siguiente sintaxis:

```
a <- as.data.frame(matrix(seq(0.5,25,0.5), nrow=10, ncol=5))
```

```
write.csv(a, "matriz1.csv", row.names=FALSE) write.csv2(a,  
"matriz2.csv", row.names=FALSE)
```

Los archivos CSV podemos leerlos con las funciones `read.csv()` o `read.csv2()`.

```
a1 <- read.csv("matriz1.csv") a2 <-  
read.csv2("matriz2.csv")
```

Datos en Excel

Es posible guardar un marco de datos en un archivo Excel utilizando la función `write.xlsx()`, disponible en la librería `xlsx`.

```
library("xlsx")
```

```
a <- as.data.frame(matrix(1:6, nrow=2, ncol=3))  
write.xlsx(a, "matriz.xls", col.names=FALSE, row.names=FALSE)
```

R es capaz de leer un archivo Excel de la versión 2003 o anteriores. Para ello utilizaremos la función `read.xls()`. Como los archivos de Excel pueden tener varias hojas de datos, debemos indicar a la función cual es el número de hoja (`sheetIndex`) que queremos leer.

Por ejemplo:

```
b = read.xlsx("matriz.xls", sheetIndex=1, header=F)
```

Manipulación de dataframes en R

En R es muy sencillo crear un nuevo data frame. Puedes unir tus variables haciendo uso de la función `data.frame` para convertir tus datos a la estructura de datos de tipo data frame. Primero, necesitas tener algunas variables almacenadas para crear el marco de datos en R. En este ejemplo vamos a definir algunas variables de datos meteorológicos. Ten en cuenta que todos los vectores tienen la misma longitud.

```
temp <- c(20.37, 18.56, 18.4, 21.96, 29.53, 28.16, 36.38, 36.62, 40.03, 27.59, 22.15,  
19.85)  
humedad <- c(88, 86, 81, 79, 80, 78, 71, 69, 78, 82, 85, 83)  
precipitaciones <- c(72, 33.9, 37.5, 36.6, 31.0, 16.6, 1.2, 6.8, 36.8, 30.8, 38.5, 22.7)
```

```
mes <- c("enero", "febrero", "marzo", "abril", "mayo", "junio",  
        "julio",    "agosto",  "septiembre", "octubre",  
        "noviembre", "diciembre")
```

Para unir los datos puedes usar la función `data.frame`. En este ejemplo, vamos a almacenar el data frame en una variable llamada `datos`:

```
datos <- data.frame(mes = mes, temperatura = temp, humedad = humedad,  
precipitaciones = precipitaciones)
```

```
names(datos) # Nombres de las variables (columnas)
```

```
“mes” “temperatura” “humedad” “precipitaciones”
```

Primero, es habitual mostrar los primeros valores para hacer algunas comprobaciones. Para ello, puedes hacer uso de la función `head` en R, que por defecto mostrará las 6 primeras filas del data frame.

```
# Primeras filas de nuestro conjunto de datos head(datos)
```

Output

```
mes  temperatura  humedad  precipitaciones
```

1	enero	20.37	88	72.0
2	febrero	18.56	86	33.9
3	marzo	18.40	81	37.5
4	abril	21.96	79	36.6
5	mayo	29.53	80	31.0
6	junio	28.16	78	16.6

En segundo lugar, puedes utilizar la función `summary` que devolverá un resumen estadístico de las variables (columnas) del conjunto de datos.

`summary(datos)`

Resumen descriptivo de los datos

```

      mes      temperatura      humedad
precipitaciones april :1    Min.    :18.40    Min.    :69.0
Min.      : 1.20 august :1          1st Qu.:20.24    1st
Qu.:78.0    1st Qu.:21.18

```

december:1 Median :24.87 Median :80.5 Median :32.45

february:1 Mean :26.63 Mean :80.0 Mean

:30.37 january :1 3rd Qu.:31.24 3rd Qu.:83.5 3rd

Qu.:36.98

Añadir columnas y filas a un data frame

A veces necesitas modificar los datos para agregar nuevas filas o columnas, o eliminarlas. Ejemplo: tenemos un dataframe (cars). La base de datos contiene 50 filas y 2 variables:

- speed: velocidad (mph).
- dist: distancia de frenado (ft).

Si usamos head(cars) obtendremos el siguiente resultado: Output of

head(cars)

speed dist

speed dist

1 4 2

2 4 10

3 7 4

4 7 22

5 8 16

6 9 10

Supongamos que queremos crear una nueva variable para transformar la velocidad en kilómetros por hora (km/h) y la distancia en metros.

Recordemos que:

kilómetro = milla/0.62137 y metro = pie/3.2808

Entonces, ahora podrías agregar dos nuevas columnas llamadas kph (kilómetros por hora) y meters (metros) con el siguiente código:

```
cars$kph <- cars$speed / 0.62137
```

```
cars$meters <- cars$dist / 3.2808
```

También puedes hacer uso de la función cbind. En caso de que quisieses agregar una nueva fila, podrías usar la función rbind.

```
kph <- cars$speed / 0.62137
```

```
meters <- cars$dist / 3.2808
```

```
cars <- cbind(cars[, c(1, 2)], kph, meters)
```


Resultando en:

Output de head(cars) con las nuevas variables speed

	dist		kph	meters
1	4	2	6.437388	0.6096074
2	4	10	6.437388	3.0480371
3	7	4	11.265430	1.2192148
4	7	22	11.265430	6.7056815
5	8	16	12.874777	4.8768593
6	9	10	14.484124	3.0480371

En resumen: Agrega nuevas filas con la función rbind y nuevas columnas con la función cbind.

Eliminar columnas y filas de un data frame

Ahora, si quieres borrar variables o filas de un data frame, tienes varias opciones:

- Usar el signo menos (-) e indicar las columnas o filas que quieras borrar.
- Crear un subconjunto de los datos que quieras conservar.

Como ejemplo, borraremos las variables speed y disty para evitar sobrescribir el conjunto de datos original guardaremos nuestros resultados en un nuevo data frame llamado cars2.

```
# Borrando la primera y segunda columna con el signo - cars2
```

```
<- cars[, -c(1, 2)]
```

```
# Seleccionar solo las columnas que queremos conservar cars2 <-
```

```
cars[, c("kph", "meters")]
```

Si vuelves a utilizar la función head, podrás ver el nuevo data frame.

```
head(cars2)
```

Output of head(cars2)

kph meters

```
1  6.437388 0.6096074
```

```
2  6.437388 3.0480371
```

```
3 11.265430 1.2192148
```

```
4 11.265430 6.7056815
```

```
5 12.874777 4.8768593
```

```
6 14.484124 3.0480371
```

Ordenando y filtrando datos de un data frame en R

Es habitual ordenar o filtrar los datos dentro de los data frames por los valores de alguna variable.

Ordenar data frames

Primero, debemos acceder al índice de ordenación de cualquier variable con la función `order`.

```
ii <- order(cars$kph) # Índice de ordenación en base a la variable 'kph'
```

El vector de índices de ordenación establece el orden en que tienen que ser elegidas las filas del data frame para obtener la ordenación deseada.

```
# Ordenando por 'kph' (de más bajo a más alto) # Solo
```

```
mostramos las primeras 4 columnas head(cars[ii, 1:4])
```

También puedes ordenar de mayor a menor haciendo uso del signo menos.

```
ii <- order(-cars$kph)
```

Filtrar data frames

Filtrar un data frame consiste en obtener una submuestra que cumpla con algunas condiciones. Para este propósito, puedes usar la función `subset` para crear subconjuntos según los valores de alguna columna.

Ejemplo:

subset(cars, khp > 12) #filtramos por la variable khp mayor a 12

También podemos crear expresiones más complejas usando los operadores lógicos

Operador lógico Descripción

& Comparación lógica 'AND' (Y) elemento a elemento

&& Comparación lógica 'AND' de vectores

| Comparación lógica 'OR' (O) elemento a elemento

|| Comparación lógica 'OR' de vectores

! Negación lógica 'NOT' (NO)

xor() Exclusión 'OR' elemento a elemento

Subconjunto donde el número de cilindros del automóvil es exactamente 6 y la potencia en caballos es mayor que 115.

Ejemplo

`subset(cars, khp > 12 & meters <6)` #filtramos con dos condiciones que deben ser

ciertas

Estadísticos descriptivos con R

Ahora que ya sabemos manejar data frames en R comenzaremos a trabajar con ellos realizando análisis estadísticos descriptivos.

En estadística, el análisis de datos es una mezcla de conocimientos estadísticos y sentido común. No existen unas pautas constantes para obtener unos resultados óptimos ya que cada base de datos es un problema diferente, con variables y objetivos diferentes. Sin embargo, cuando se abarca un análisis de cualquier base de datos es obligatorio estudiar la estructura de los datos, un análisis descriptivo, tanto analítico como gráfico, de los datos, etc.

El análisis exploratorio de datos es fundamental en cualquier estudio estadístico, por encima de análisis de aprendizaje de modelos estadísticos que sean capaces de predecir valores futuros, ya que si este paso no se hace con cautela no se tendrán datos de calidad y, por tanto, el modelo aprenderá de forma ineficiente.

En esta clase tomaremos un primer contacto con las técnicas estadísticas que se utilizan para ordenar, analizar y representar un conjunto de datos, con el fin de describir apropiadamente sus características. El primer paso en el análisis de datos, una vez introducidos los mismos, es realizar un análisis descriptivo o análisis exploratorio de datos. Los principales objetivos de un análisis descriptivo o análisis exploratorio de datos son la descripción y la síntesis de los datos. Para ello, los datos se organizan en tablas, se calculan medidas que describen sus características más importantes y se realizan representaciones gráficas.

Este análisis inicial proporciona una idea de la forma que tiene la distribución de las observaciones y permite obtener estadísticos de tendencia central (media, mediana y moda), de dispersión (varianza, desviación típica, rango), de forma (asimetría, curtosis), de posición (percentiles), así como gráficos de barras, de sectores e histograma.

R y RStudio disponen de un amplio conjunto de herramientas para el análisis descriptivo de un conjunto de datos.

Tabla de Frecuencias

Para crear tablas de frecuencia en R y R Studio se emplea la función `table` o la función `prop.table`, dependiendo de si la tabla muestra las frecuencias absolutas o las frecuencias relativas. La sintaxis de estas órdenes es la siguiente:

> **`table(x)` # para frecuencias absolutas**

> **`prop.table(tab)` # para las frecuencias relativas**

La principal diferencia entre las dos funciones reside en el tipo de los argumentos que necesita cada una.

- `table` construye la tabla de frecuencias absolutas a partir de la variable que recibe como argumento
- `prop.table` recibe como argumento una tabla o una matriz que representa una tabla de frecuencias absolutas, y a partir de ella construye la tabla de frecuencias relativas asociada. Es decir, `prop.table` recibe como argumento el resultado que devuelve la función `table`.

3. Instancia de evaluación: Participación en foro obligatorio

En esta instancia de evaluación deberás elegir un dataset de tu interés y aplicar las distintas funciones vistas en esta clase para importar y manipular el dataset en un dataframe y realizar un estudio estadístico del mismo. También podés modificar el dataset creando nuevas columnas, ordenando y filtrando para practicar.

En el foro deberás subir un archivo comprimido (zip o rar) que incluya el script .r con el código que escribiste para resolver la actividad y un archivo PDF con un pequeño informe sobre los resultados obtenidos para compartirlo con el resto de los estudiantes.

4. Cierre

Como cierre de esta clase te invito a que compares lo que has aprendido en otros cursos sobre el lenguaje Python y qué tiene en común con el lenguaje R, sus distintas aplicaciones y cuándo es mejor usar uno u el otro.

Te esperamos en los próximos encuentros. Recordá que es importante participar de los foros, de los encuentros sincrónicos y en lo posible de las tutorías presenciales.

5. Bibliografía Obligatoria:

Santana J., & Efraín M. (2014) El arte de programa en R: un lenguaje para la estadística / Julio Sergio. México Instituto Mexicano de Tecnología del Agua. UNESCO. Comité Nacional Mexicano del Programa Hidrológico Internacional.

Ximénez, C., & De Rivera Y Revuelta, J. L. G. (2022). Análisis de datos en lenguaje R. <https://doi.org/10.15366/9788483448304.dt.107>

Paradis, E. (2003) R para Principiantes

